

TD1 - PRAM

Conventions. Arrays are indexed from 1. The *time* of an algorithm is its parallel running time on the PRAM, using the number of PUs it is given. Its *work*, cost, work-optimality and efficiency are defined as in the lecture, with respect to the time T_{seq} of the best sequential algorithm for the same problem. NIL denotes the null pointer. You may call the algorithms seen in class as building blocks: BCAST(e, N) returns an array $pe[1 \dots N]$ with $pe[i] = e$ for all i , in $O(\log_2 N)$ time on an EREW PRAM with N PUs; REDUCE($A[1 \dots N], \otimes$) returns $A[1] \otimes \dots \otimes A[N]$ for an associative operation $\otimes$, in $O(\log_2 N)$ time on an EREW PRAM with N PUs; RANKCOMPUTATION($next[1 \dots N]$) returns the array $d[1 \dots N]$ of the distances from every node to the end of the list, in $O(\log_2 N)$ time on an EREW PRAM with N PUs.

Exercise 1

Is the array sorted?

Let $A[1 \dots N]$ be an array of N integers.

- Design an algorithm ISORTED($A[1 \dots N]$) that returns *true* if $A[1] \leq A[2] \leq \dots \leq A[N]$ and *false* otherwise, in $O(1)$ time on a CRCW PRAM with N PUs. List all the concurrency modes that can be employed for the writes of this algorithm.
- Verify the reads of your algorithm, by decomposing them into elementary steps as was done for list ranking in the lecture: are they exclusive? Does the algorithm need concurrent reads, or only concurrent writes?
- Design an EREW PRAM version of ISORTED. What are its time, work and cost? Is it work-optimal? Is it efficient?

Exercise 2

From a list to a ring

Let $next[1 \dots N]$ be a linked list of N nodes: $next[i]$ is the successor of node i , and $next[\ell] = \text{NIL}$ for the last node ℓ . The head h of the list is the only node that no other node points to. We want to close the list into a ring, by making the last node point to the head: $next[\ell] \leftarrow h$.

- Design an EREW PRAM algorithm LISTTORING($next[1 \dots N]$) that updates $next[1 \dots N]$ into a ring and returns the index h of the head. Verify that all reads and writes are exclusive.
- What are its time, work and cost? Is it work-optimal? Is it efficient?

Exercise 3

Inverting a list

Let $next[1 \dots N]$ be a linked list of N nodes.

- Design an EREW PRAM algorithm INVERT($next[1 \dots N]$) that computes the inverse of this linked list, and returns it in a pointer array $prev[1 \dots N]$ (e.g., if $next[3] = 7$, then $prev[7] = 3$).
- What are its time, work and cost? Is it work-optimal? Is it efficient?
- Now let $next[1 \dots N]$ be a ring (no pointer is NIL), and h the index of one of its nodes. Design an EREW PRAM algorithm RINGTOLIST($next[1 \dots N], h$) that turns the ring into a list whose head is h . What is its time?

Exercise 4

Work-optimal and efficient broadcast and reduction

In the lecture, BCAST(e, N) runs in $O(\log_2 N)$ time on an EREW PRAM with N PUs, but it creates N PUs at each of its $\lceil \log_2 N \rceil$ iterations: its work and its cost are both $O(N \log_2 N)$, whereas $T_{seq}(N) = O(N)$.

- Design a version of BCAST(e, N) that still uses up to N PUs and runs in $O(\log_2 N)$ time, but is work-optimal. Is it efficient?
- Now design a version BCASTP(e, N, P) that broadcasts e into $pe[1 \dots N]$ using only P PUs, with $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

- c) Do the same for the reduction. The recursive REDUCE of the lecture is already work-optimal: first write a non-recursive (iterative) version of the reduction, in the style of BCAST, that uses up to N PUs and is work-optimal. Then design $\text{REDUCEP}(A[1 \dots N], \otimes, P)$ that computes the same reduction with P PUs, $P \ll N$. What are its time, work and cost, as functions of N and P ? For which values of P does the algorithm stay efficient?

Exercise 5

Prefix scan on an array

Let $X[1 \dots N]$ be an array of size N and $\otimes$ an associative operation.

- a) Design an EREW PRAM algorithm $\text{SCAN}(X[1 \dots N], \otimes)$ that computes and returns the prefix scan array $X'[i] = X[1] \otimes \dots \otimes X[i]$ for all i , using N PUs and running in $O(\log_2 N)$ time. Verify that all reads and writes are exclusive. Do not treat the array as a linked list (Hint: Try using recursion on the array). Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- b) This time, design an algorithm specifically for P PUs that does the same computation in $O(N/P + \log_2 P)$ time, then compute its work and cost. Is it work-optimal? For what values of P does it stay efficient?

Exercise 6

Prefix scan on a list

Let $\text{next}[1 \dots N]$ be a linked list of N nodes ($\text{next}[i] = \text{NIL}$ for the last node), $x[1 \dots N]$ a value attached to each node, and $\otimes$ an associative operation. In the lecture, the list ranking algorithm computed, with pointer jumping, the distance $d[i]$ from every node i to the end of the list.

- a) Design an algorithm $\text{SCAN}(\text{next}[1 \dots N], x[1 \dots N], \otimes)$ that computes and returns the array $y[1 \dots N]$ where $y[i] = x[i] \otimes x[\text{next}[i]] \otimes \dots \otimes x[\ell]$ is the combination of the values from node i to the last node ℓ of the list, in $O(\log_2 N)$ time using N PUs. Hint: start from the list ranking algorithm of the lecture.
- b) Verify the memory accesses of your algorithm: are all reads exclusive? all writes? On which PRAM model does it run as written, and how would you modify it to run on an EREW PRAM?
- c) Compute the work and cost of this algorithm. Is it work-optimal? Is it efficient?
- d) Show that list ranking is a particular case of SCAN: for which values $x[i]$ and which operation $\otimes$ does SCAN compute the distance $d[i]$ to the end of the list?
- e) Using a single call to SCAN, design an algorithm $\text{RANKBYPREFIX}(\text{next}[1 \dots N])$ that computes and returns the rank of every node from the head of the list (0 for the head, $N - 1$ for the last node).
- f) Generalize: how would you compute, for every node i , the combination of the values from the *head* of the list to i (the prefix in the usual direction) in the same time? Pay attention to the order of the operands if $\otimes$ is not commutative.

Exercise 7

Selection in a list on an EREW PRAM

Let $\text{next}[1 \dots N]$ be a linked list of N nodes, each node i having a label $\text{label}[i] \in \{\text{red}, \text{blue}\}$.

- a) Design an EREW PRAM algorithm $\text{SELECTBLUES}(\text{next}[1 \dots N], \text{label}[1 \dots N])$ that updates the list $\text{next}[1 \dots N]$ so that in the end i) each blue node points to the next blue node in the original list, ii) each red node points to NIL, and iii) the algorithm returns the index of the head node of the updated list containing all blue nodes.
- b) What are its time, work and cost?

Exercise 8

Separation in an array on an EREW PRAM

Let $A[1 \dots N]$ be an array of N objects with an associated function $\text{LABEL}(A[i]) = 0$ or $\text{LABEL}(A[i]) = 1$ for its elements.

- a) Design an EREW PRAM algorithm $\text{SEPARATE01}(A[1 \dots N])$ that returns a permuted copy $A'[1 \dots N]$ of this array containing all elements with label 0 in the beginning of the array (so that the labels of $A'[1], \dots, A'[N]$ read $0, \dots, 0, 1, \dots, 1$). Hint: Try to use the prefix scan algorithm to determine where to write each element after permutation.

- b) What are its time, work and cost?

Exercise 9

Sorting an array of unique integers

Let $X[1 \dots N]$ be an array of unique integers, meaning no two elements $X[i]$ and $X[j]$ are equal for $1 \leq i \neq j \leq N$.

- Design a CRCW PRAM algorithm $\text{SORT}(X[1 \dots N])$ that returns the sorted array $X[1 \dots N]$. Specify the concurrency mode used for the concurrent writes.
- What are its time, work and cost?
- How can you make the algorithm work if the elements of $X[1 \dots N]$ are not unique?
- How can you make the algorithm work on a CREW PRAM?

Exercise 10

Parallel matrix multiplication

The multiplication of two square matrices $A, B \in \mathbb{R}^{N \times N}$ yields another square matrix $C = AB \in \mathbb{R}^{N \times N}$ whose elements C_{ij} are defined as follows:

$$C_{ij} = \sum_{k=1}^N A_{ik} B_{kj}, \quad 1 \leq i, j \leq N$$

- Design a CRCW PRAM algorithm $\text{MATMUL}(A, B)$ that carries out the multiplication of two $N \times N$ matrices $C = AB$, then returns the matrix C . What are its time, work and cost? To obtain this time, how many PUs are needed?
- Do the same for a CREW PRAM. What are its time, work and cost? How can you make this algorithm work on an EREW PRAM as well?

Exercise 11

Simulation of concurrency modes on a CRCW PRAM

Consider a CRCW PRAM in *consistent* mode: simultaneous writes to the same memory cell are allowed only if they all write the same value.

- Prove that a CRCW PRAM in consistent mode can simulate a CRCW PRAM in *fusion mode* with $\min$ (a cell written simultaneously by several PUs receives the minimum of the written values), with the same time up to a constant factor. The simulation may use more PUs than the simulated algorithm. Hint: with k^2 PUs, compare all the pairs among the k values written to a cell, and mark every value that is larger than another one; which concurrency mode does the marking need?

Exercise 12

Computing the depth of nodes in a tree

Let $\text{parent}[1 \dots N]$ be the array of parent pointers in a tree having N nodes (such that $\text{parent}[i] = \text{NIL}$ if the node i is the root of the tree, and $1 \leq \text{parent}[i] \neq i \leq N$ otherwise). The goal of this exercise is to compute the depth of all nodes in the tree in parallel. The depth of the root node is defined to be zero, and the depth of any other node is the depth of its parent plus one (i.e., $\text{depth}[i] = \text{depth}[\text{parent}[i]] + 1$).

- Design a CREW PRAM algorithm $\text{TREEDEPTH}(\text{parent}[1 \dots N])$ to compute the depth of each node in the tree, and return it in an array. What are its time, work and cost? Would your algorithm work on an EREW PRAM?
- This time, assume that the tree is binary, and in addition to parent pointers, each node i also has a pointer towards its left and right children $\text{left}[i]$ and $\text{right}[i]$ (with $\text{left}[i] = \text{NIL}$ if the node i has no left child, likewise for the right child). Design an EREW PRAM algorithm $\text{BINTREEDEPTH}(\text{parent}[1 \dots N], \text{left}[1 \dots N], \text{right}[1 \dots N])$ to compute and return the depth of all nodes in the tree in $O(\log_2 N)$ time. Hint: Consider the depth-first traversal of the tree, and try to use the prefix scan algorithm on this traversal.

Exercise 13

Parallel breadth-first search

The goal of this exercise is to perform a parallel breadth-first search (BFS) on a given directed graph $G(V, E)$ having $|V|$ vertices and $|E|$ edges $E[t] = (u_t, v_t)$ for $1 \leq t \leq |E|$, with $1 \leq u_t \neq v_t \leq |V|$.

- a) Design a CRCW PRAM algorithm $\text{BFS}(G(V, E), s)$ that performs a parallel BFS on a given directed graph G , starting from the vertex s ($1 \leq s \leq |V|$). The algorithm should compute and return two arrays: $\text{parent}[i]$, the parent of each vertex i in the BFS tree, and $\text{level}[i]$, the level of vertex i in the BFS tree (i.e., its distance from the starting vertex s). Specify the concurrency mode to be employed.
- b) What are its time, work and cost?

Exercise 14

Connected components of an undirected graph

Let $G(V, E)$ be an undirected graph with $|V|$ vertices and $|E|$ edges $E[t] = (u_t, v_t)$ for $1 \leq t \leq |E|$, with $1 \leq u_t \neq v_t \leq |V|$. We want to label every vertex i with $C[i]$, the smallest vertex index of the connected component that i belongs to. We will use a simple algorithm: start with $C[i] = i$ for every vertex, then let every edge *merge* its two ends in parallel, by giving both of them the smaller of their two labels; repeat until no edge changes anything, i.e., until both ends of every edge carry the same label.

- a) Design a CRCW PRAM algorithm $\text{CONNECTEDCOMPONENTS}(V, E)$ that follows this idea, using one PU per edge. Specify the concurrency modes: for the writes to the labels, and for the test that stops the loop.
- b) Show that the algorithm terminates and that the labels are correct in the end. Hint: let m be the smallest vertex of a component; after k iterations, which vertices of the component are guaranteed to carry the label m ?
- c) What are its time, work and cost, as functions of $|V|$, $|E|$ and the largest diameter D of a component (the diameter is the largest distance between two vertices of the component)? Is it work-optimal? Is it efficient?
- d) Give a family of graphs with $|V|$ vertices on which the algorithm needs $\Theta(|V|)$ iterations. (The optimal $O(\log |V|)$ algorithms for this problem combine this idea with pointer jumping on the labels; they are beyond the scope of this course.)